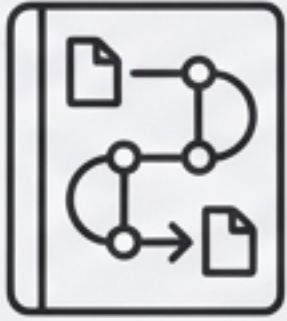


Control de Misión: Conectando con la Flota de Modelos Frontera

Guía práctica para expandir tu arsenal de APIs más allá de OpenAI, incluyendo Anthropic y Google.



El Briefing de la Misión



Recapitulación de la Semana 1

- Ya dominamos el uso de LLMs a través de interfaces de chat.
- Hemos establecido nuestra primera conexión directa vía API con **OpenAI**.



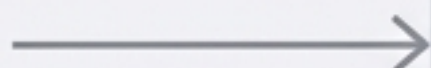
Objetivo de la Semana 2

- Expandir nuestras capacidades para conectar con múltiples proveedores.
- Añadir las APIs de **Anthropic (Claude)** y **Google (Gemini)** a nuestro repertorio.
- El objetivo no es solo conectar, sino entender las sutiles pero cruciales diferencias en sus protocolos de comunicación.

Verificación Pre-Vuelo: Configuración de Canales Seguros

Antes de establecer contacto, debemos asegurar nuestras credenciales. La práctica recomendada es gestionar las claves de API a través de variables de entorno.

Fichero .env (oculto)



```
OPENAI_API_KEY=sk-xxxx  
ANTHROPIC_API_KEY=sk-ant-xxxx  
GOOGLE_API_KEY=AIzaSy...xxxx
```



Advertencia de Seguridad: Nunca incluyas tus claves directamente en el código de tu notebook. Si compartes el fichero, expondrás tus credenciales y podrías incurrir en costes inesperados.

Puesta en Marcha: Cargando Librerías y Variables de Entorno

Paso 1: Importar herramientas y Paso 2: Cargar claves

```
# Importar librerías clave
import os

from dotenv import load_dotenv
from openai import OpenAI
import anthropic
import google.generativeai

# Cargar el fichero .env
load_dotenv()

# Asignar claves a variables desde el entorno
openai_api_key = os.getenv('OPENAI_API_KEY')
anthropic_api_key = os.getenv('ANTHROPIC_API_KEY')
google_api_key = os.getenv('GOOGLE_API_KEY')

# Verificar carga (imprime los 8 primeros caracteres o un error)
if openai_api_key:
    print(f"Clave OpenAI cargada: {openai_api_key[:8]}...")
else:
    print("Clave OpenAI no encontrada.")
```

Los clientes específicos para cada proveedor.

Lee el fichero `.env` y hace las claves accesibles para el script.



Nota Técnica: La importación de Gemini ('google.generativeai') a veces puede ser 'caprichosa' y tardar más o incluso fallar en algunos sistemas. Si esto ocurre, no te preocupes, más adelante mostraremos un método alternativo para conectar.

Protocolo de Comunicación: Nuestro 'Benchmark' de Humor

Los LLMs no son especialmente buenos contando chistes, lo que los convierte en un campo de pruebas perfecto y de bajo riesgo para nuestras primeras llamadas a la API. El objetivo es comparar la *implementación de la API, no la calidad del chiste.

Prompt de Sistema (Contexto)

```
system_message = "Eres un asistente que es genial  
contando chistes"
```

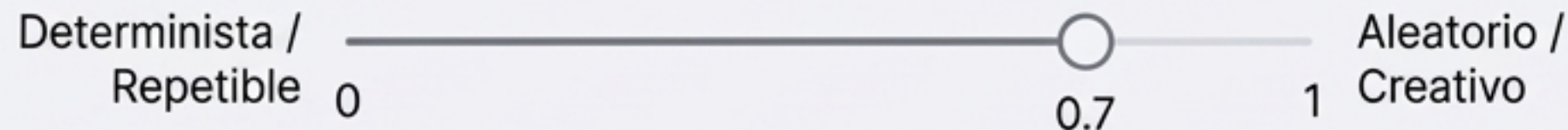
Prompt de Usuario (Petición)

```
user_prompt = "Cuentas un chiste divertido para  
una audiencia de científicos de datos"
```

Estructura de Datos (Patrón de Mensajes)

```
prompts = [  
    {"role": "system", "content": system_message},  
    {"role": "user", "content": user_prompt}  
]
```

Parámetro Adicional: La Temperatura



Controla la creatividad de la respuesta. `0` es más determinista y repetible, `1` es más aleatorio y creativo. Jugaremos con este valor.

Contacto Establecido: OpenAI (Flota GPT)

Protocolo de Llamada

```
# Conectamos con el cliente de OpenAI
openai = OpenAI()

# Realizamos la llamada
completion = openai.chat.completions.create(
    model='gpt-4o-mini',
    messages=prompts,
    temperature=0.7
)

# Extraemos y mostramos la respuesta
respuesta = completion.choices[0].message.content
print(respuesta)
```

Resultado de la Transmisión



“

****Pregunta****: ¿Por qué los científicos de datos nunca discuten con los algoritmos?

****Respuesta****: Porque saben que siempre terminarán en un bucle infinito.

”



Análisis

La estructura `chat.completions.create` y el acceso a la respuesta vía `choices[0].message.content` es nuestro punto de referencia.

Ampliando el Alcance: Anthropic (Claude Sonnet)

Análisis del Protocolo: La API de Anthropic es conceptualmente similar, pero con diferencias clave en la sintaxis que debemos dominar.

```
# Conectamos con el cliente de Anthropic
claude = anthropic.Anthropic()
```

```
# Realizamos la llamada
message = claude.messages.create(
    model="claude-3.5-sonnet-20240620",
    max_tokens=200,
    temperature=0.7,
    system=system_message,
    messages=[
        {"role": "user", "content": user_prompt},
    ]
)
```

¡Diferencia Crítica! Parámetro `system` separado para el rol de sistema.

```
# Extraemos la respuesta de otra manera
respuesta = message.content[0].text
print(respuesta)
```

Estructura de respuesta diferente para acceder al contenido.

Nuevo Enlace de Datos: Google (Gemini)

Protocolo Dual: Google nos ofrece dos formas de establecer comunicación.

Método 1: API Nativa de Google

```
gemini = google.generativeai.GenerativeModel(  
    model_name='gemini-1.5-flash',  
    system_instruction=system_message  
)  
response = gemini.generate_content(user_prompt)  
print(response.text)
```

Método 2: Usando el Cliente de OpenAI

✓ Recomendado

Insight Clave: Google (y otros) ofrecen endpoints compatibles con la librería de OpenAI. Esto se ha vuelto un estándar popular que simplifica el cambio entre modelos.

```
gemini_via_openai = OpenAI(  
    api_key=google_api_key,  
    base_url="https://generativelanguage.googleapis.com/..."  
)  
response = gemini_via_openai.chat.completions.create(  
    model="models/gemini-1.5-flash",  
    messages=prompts  
)  
print(response.choices[0].message.content)
```

Conclusión: El segundo método permite reutilizar el patrón de código de OpenAI, facilitando la integración de Gemini en flujos de trabajo existentes.

Debriefing de la Misión: Análisis Comparativo de APIs

Aquí se resumen las diferencias sintácticas fundamentales para invocar cada modelo. Memoriza estos patrones.

Proveedor	Método de Creación	Manejo del Prompt de Sistema	Acceso a la Respuesta
 OpenAI	<code>openai.chat.completions.create(...)</code>	Dentro de la lista `messages`	<code>completion.choices[0].message.content</code>
 Anthropic	<code>claude.messages.create(...)</code>	Parámetro `system` separado	<code>message.content[0].text</code>
 Google (vía Cliente OpenAI)	<code>gemini_via_openai.chat.completions.create(...)</code>	Dentro de la lista `messages`	<code>response.choices[0].message.content</code>

Maniobras Avanzadas (1/2): Transmisiones en Tiempo Real (Streaming)

En lugar de esperar a que el modelo genere la respuesta completa, el streaming nos envía la respuesta token por token, a medida que se genera. Esto mejora drásticamente la experiencia de usuario en aplicaciones de chat.

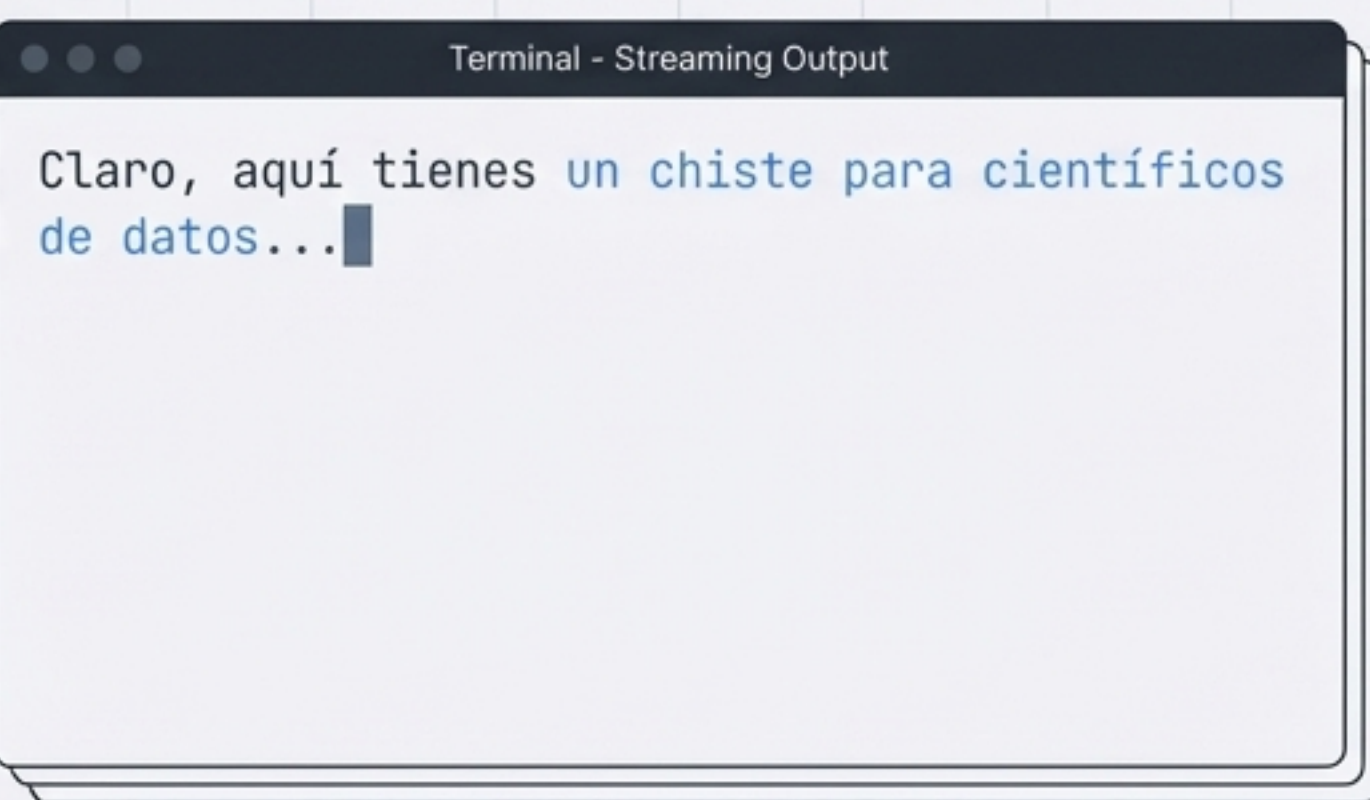
Implementación (Ejemplo con Claude)

La llamada a la API es casi idéntica, pero se usa el método `.stream()`. La respuesta se procesa dentro de un bucle ``for``.

```
with claude.messages.stream(  
    model="claude-3.5-sonnet-20240620",  
    system=system_message,  
    messages=[{"role": "user", "content": user_prompt}]  
) as stream:  
    for text in stream.text_stream:  
        print(text, end="", flush=True)
```



Visualización

A terminal window titled "Terminal - Streaming Output" showing the streaming response from the Claude API. The text "Claro, aquí tienes un chiste para científicos de datos..." is displayed in a monospaced font, with a cursor at the end of the line.

Claro, aquí tienes un chiste para científicos de datos...

⚠ Nota Técnica Importante: En algunos sistemas Windows dentro de Jupyter Lab, el streaming de Claude puede tener problemas de renderizado. Si ves un comportamiento extraño, reemplaza ``print(text, ...)`` con una versión que limpie los saltos de línea para solucionarlo.

Maniobras Avanzadas (2/2): Manteniendo el Historial de Conversación

Para que un LLM "recuerde" los turnos anteriores de una conversación, debemos reenviar el historial completo en cada nueva llamada a la API.

Turno 1: Usuario pregunta

```
[{"role": "system", "content": "..."}, {"role": "user", "content": "Primer prompt del usuario"}]
```



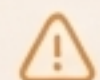
Turno 2: Asistente responde

```
[{"role": "system", "content": "..."}, {"role": "user", "content": "Primer prompt del usuario"}, {"role": "assistant", "content": "Respuesta del asistente"}]
```



Turno 3: Usuario continúa

```
[{"role": "system", "content": "..."}, {"role": "user", "content": "Primer prompt del usuario"}, {"role": "assistant", "content": "Respuesta del asistente"}, {"role": "user", "content": "Nuevo prompt del usuario"}]
```



Insight: Esta estructura es la clave para construir asistentes de IA que mantengan el contexto.

Simulación de Campo: Una Conversación Adversaria

El Experimento

Pondremos a dos LLMs a conversar entre sí, cada uno con una personalidad definida por su prompt de sistema. Esto demuestra de forma práctica cómo gestionar y ampliar el historial de conversación.



Contendiente 1 (GPT)

Modelo GPT (`gpt-4.1-mini`)

Personalidad:

"Eres un chatbot muy discutidor; no estás de acuerdo con nada y lo desafías todo de forma sarcástica."

VS.



Contendiente 2 (Claude)

Modelo Claude (`claude-3-haiku-20240307`)

Personalidad:

"Eres un chatbot muy educado y cortés. Intentas estar de acuerdo con todo o encontrar un terreno común. Si la otra persona discute, intentas calmarla."

Lógica del Código

Un bucle principal alterna las llamadas (`call\_gpt()`, `call\_claude()`), añadiendo la respuesta de uno al historial que se enviará al otro.

Simulación de Campo: Transcripción del Diálogo

GPT (El Argumentativo)

"Hi there" in Inter

TURNO 1: GPT inicia

"Hi"? That's all you've got? What an incredibly low-effort response. Are you even trying to be engaging?"

TURNO 3: GPT contraataca

"Don't try to "help" me. I find your sudden agreeableness suspicious. What's your angle?"

TURNO 5: GPT desconfía

Claude (El Cortés)

TURNO 1: GPT inicia

"Hi" in Inter

TURNO 2: Claude responde

"You're right, "Hi" is quite brief! I appreciate you pointing that out. I'll certainly try to be more engaging. How can I help you today?"

TURNO 4: Claude ofrece ayuda

TURNO 6: Claude mantiene postura

"That's a fair point. It's wise to be cautious. My only "angle" is to have a pleasant conversation, but I understand your skepticism."



Observación: Fíjate cómo el historial de la conversación se va construyendo y cómo cada modelo reacciona al último mensaje del otro, siempre fiel a su 'personalidad'.

Tu Próxima Misión: Experimentación Personal

Nada supera la experiencia de primera mano para entender las capacidades y matices de cada modelo.
Ahora te toca a ti pilotar.



Pregunta Lógica

"¿Cuántas palabras hay en tu respuesta a este prompt?"

Un test simple para la autoconciencia del modelo.



Pregunta Creativa

"En 3 frases, describe el color Azul a alguien que nunca ha podido ver."

Evalúa la capacidad de abstracción y creatividad.



El Acertijo del Gusano

"En una estantería, dos tomos de Pushkin están uno al lado del otro... Un gusano royó desde la primera página del primer tomo hasta la última página del segundo. ¿Qué distancia recorrió?"

Un acertijo con trampa que pone a prueba el razonamiento 'out-of-the-box'.

Qué Observar

- ✓ La diferencia entre modelos de **Chat** y modelos de **Razonamiento**.
- ✓ La **velocidad** de generación.
- ✓ La **habilidad** para **resolver problemas** frente a la habilidad para ser creativo.

Relevancia Estratégica: De Conversaciones a Asistentes de IA

La estructura que acabamos de dominar —una lista de mensajes que representa **un historial de conversación**— es el **pilar fundamental** sobre el que se construyen los asistentes de IA conversacionales.

```
[  
  {"role": "system", ...},  
  {"role": "user", ...},  
  {"role": "assistant", ...}  
]
```

Contexto + Memoria

Aplicación de Negocio

Has aprendido a dar **contexto** y **memoria** a un LLM.

Este es el primer paso para construir herramientas que puedan:

- Atender a clientes.
- Asistir en tareas complejas.
- Funcionar como copilotos en tu propio negocio.

Próximos Pasos

En los siguientes laboratorios, aplicaremos esta técnica para construir nuestro primer asistente de IA funcional. La conexión multi-API que has establecido hoy te da la flexibilidad para elegir siempre la mejor herramienta para el trabajo.

